Uber Trip Analysis - Exploratory Data Analysis (EDA)

Project Overview

This project performs an Exploratory Data Analysis (EDA) on Uber trip data to understand customer booking patterns, trip behavior, payment preferences, vehicle demand, and location-based insights.

The analysis is performed using **Python**, **Pandas**, **NumPy**, **Matplotlib**, **Seaborn**, and **Plotly**, followed by an interactive **Power BI Dashboard** for business reporting.

Objectives

- Understand the structure of the dataset
- Perform data cleaning and preprocessing
- Analyze booking trends
- Study payment preferences
- Explore trip distance and fare distribution
- Identify popular pickup and drop-off locations
- Generate business insights and recommendations

```
# Import Libraries

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

import plotly.express as px
import plotly.graph_objects as go

import warnings
warnings.filterwarnings("ignore")

plt.style.use("ggplot")

print("Libraries Imported Successfully ✅")
```

```
Libraries Imported Successfully ✅
```

```
# Load Excel Files

trip_df = pd.read_excel("Uber Trip Details.xlsx")
location_df = pd.read_excel("Location Table.xlsx")
```

```
trip_df.head()
```

```
location_df.head()
```

```
print("Trip Dataset Shape :", trip_df.shape)
print("Location Dataset Shape :", location_df.shape)
```

```
trip_df.columns
```

```
Index(['Trip ID', 'Pickup Time', 'Drop Off Time', 'passenger_count',
       'trip_distance', 'PULocationID', 'DOLocationID', 'fare_amount',
       'Surge Fee', 'Vehicle', 'Payment_type'],
      dtype='object')
```

```
location_df.columns
```

```
trip_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103728 entries, 0 to 103727
Data columns (total 11 columns):
 #   Column           Non-Null Count   Dtype
---  ------           --------------   -----
 0   Trip ID          103728 non-null  int64
 1   Pickup Time      103728 non-null  datetime64[ns]
 2   Drop Off Time    103728 non-null  datetime64[ns]
```

```
 3   passenger_count  103728 non-null  int64
 4   trip_distance    103728 non-null  float64
 5   PULocationID     103728 non-null  int64
 6   DOLocationID     103728 non-null  int64
 7   fare_amount      103728 non-null  float64
 8   Surge Fee        103728 non-null  float64
 9   Vehicle          103728 non-null  object
 10  Payment_type     103728 non-null  object
dtypes: datetime64[ns](2), float64(3), int64(4), object(2)
memory usage: 8.7+ MB
```

```
location_df.info()
```

```
# Missing Values

trip_df.isnull().sum()
```

```
location_df.isnull().sum()
```

```
print("Duplicate rows in Trip Dataset :", trip_df.duplicated().sum())
print("Duplicate rows in Location Dataset :", location_df.duplicated().sum())
```

```
trip_df.describe()
```

```
trip_df.describe(include='object')
```

```
print("="*60)
print("TRIP DATASET INFORMATION")
print("="*60)

print(f"Rows    : {trip_df.shape[0]}")
print(f"Columns : {trip_df.shape[1]}")

print("\nMissing Values")
print(trip_df.isnull().sum())

print("\nDuplicate Rows")
print(trip_df.duplicated().sum())
```

```
============================================================
TRIP DATASET INFORMATION
============================================================
Rows    : 103728
Columns : 11

Missing Values
Trip ID            0
Pickup Time        0
Drop Off Time      0
passenger_count    0
trip_distance      0
PULocationID       0
DOLocationID       0
fare_amount        0
Surge Fee          0
Vehicle            0
Payment_type       0
```

```
dtype: int64

Duplicate Rows
0
```

Data Quality Assessment

Observations

- The **Trip Details** dataset contains **103,728 rows** and **11 columns**, while the **Location** dataset contains **265 rows** and **3 columns**.
- The **Trip Details** dataset has **no missing values**, indicating that all trip-related records are complete.
- The **Location** dataset contains **1 missing value in the `Location` column** and **2 missing values in the `City` column**, which will need to be addressed during preprocessing.
- No duplicate records were found in either dataset.
- The data types are appropriate, with datetime fields for pickup and drop-off times, numerical fields for trip metrics, and categorical fields for vehicle and payment information.
- Overall, the datasets are well-structured and suitable for exploratory data analysis after handling the small number of missing values in the Location dataset.

```
# Create a copy of the original dataset
df = trip_df.copy()
```

```
df['Pickup Date'] = df['Pickup Time'].dt.date
```

```
df['Pickup Hour'] = df['Pickup Time'].dt.hour
```

```
df['Pickup Day'] = df['Pickup Time'].dt.day_name()
```

```
df['Pickup Month'] = df['Pickup Time'].dt.month_name()
```

```
df['Weekday Number'] = df['Pickup Time'].dt.weekday
```

```
df['Day Type'] = np.where(
    df['Weekday Number'] >= 5,
    'Weekend',
    'Weekday'
)
```

```
df['Trip Duration (min)'] = (
    df['Drop Off Time'] - df['Pickup Time']
).dt.total_seconds() / 60
```

```
df['Trip (Day/Night)'] = np.where(
    (df['Pickup Hour'] >= 6) & (df['Pickup Hour'] < 18),
    'Day Trip',
    'Night Trip'
)
```

```
df.head()
```

```
# Merge Pickup Location

df = df.merge(
    location_df,
    left_on="PULocationID",
    right_on="LocationID",
    how="left"
)
```

```
df.rename(columns={
    "Location": "Pickup Location",
    "City": "Pickup City"
}, inplace=True)
```

```
df.drop(columns=["LocationID"], inplace=True)
```

```
df = df.merge(
    location_df,
```

```
    left_on="DOLocationID",
    right_on="LocationID",
    how="left"
)
```

```
df.rename(columns={
    "Location": "Dropoff Location",
    "City": "Dropoff City"
}, inplace=True)
```

```
df.drop(columns=["LocationID"], inplace=True)
```

```
df.head()
```

```
df.columns
```

Data Integration

Objective

The Trip Details dataset was merged with the Location dataset using Pickup Location ID and Drop-off Location ID. This transformation replaced numerical location identifiers with meaningful location names and cities, making the analysis easier to interpret and more business-friendly.

Columns Added

- Pickup Location
- Pickup City
- Dropoff Location
- Dropoff City

```
# ============================================================
# Merge Trip Dataset with Location Dataset
# ============================================================

# Pickup Location
trip_df = trip_df.merge(
    location_df,
    left_on='PULocationID',
    right_on='LocationID',
    how='left'
)

trip_df.rename(columns={
    'Location':'Pickup Location',
    'City':'Pickup City'
}, inplace=True)

trip_df.drop('LocationID', axis=1, inplace=True)


# Dropoff Location
```

```
trip_df = trip_df.merge(
    location_df,
    left_on='DOLocationID',
    right_on='LocationID',
    how='left'
)

trip_df.rename(columns={
    'Location':'Dropoff Location',
    'City':'Dropoff City'
}, inplace=True)

trip_df.drop('LocationID', axis=1, inplace=True)

print("Datasets merged successfully.")
```

```
trip_df.head()
```

```
trip_df.shape
```

```
trip_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103728 entries, 0 to 103727
Data columns (total 11 columns):
 #   Column           Non-Null Count   Dtype
---  ------           --------------   -----
 0   Trip ID          103728 non-null  int64
 1   Pickup Time      103728 non-null  datetime64[ns]
 2   Drop Off Time    103728 non-null  datetime64[ns]
 3   passenger_count  103728 non-null  int64
 4   trip_distance    103728 non-null  float64
 5   PULocationID     103728 non-null  int64
 6   DOLocationID     103728 non-null  int64
 7   fare_amount      103728 non-null  float64
 8   Surge Fee        103728 non-null  float64
 9   Vehicle          103728 non-null  object
 10  Payment_type     103728 non-null  object
dtypes: datetime64[ns](2), float64(3), int64(4), object(2)
memory usage: 8.7+ MB
```

```
# ============================================================
# Feature Engineering
# ============================================================

# Date
trip_df['Pickup Date'] = trip_df['Pickup Time'].dt.date

# Time
trip_df['Pickup Hour'] = trip_df['Pickup Time'].dt.hour

# Month
trip_df['Month'] = trip_df['Pickup Time'].dt.month_name()

# Day Name
trip_df['Day Name'] = trip_df['Pickup Time'].dt.day_name()

# Weekday Number
trip_df['Weekday'] = trip_df['Pickup Time'].dt.weekday

# Weekend / Weekday
trip_df['Trip Day'] = np.where(
    trip_df['Weekday'] >= 5,
    'Weekend',
    'Weekday'
)

# Trip Duration (minutes)
trip_df['Trip Duration (min)'] = (
    trip_df['Drop Off Time'] - trip_df['Pickup Time']
).dt.total_seconds() / 60

# Day / Night Trip
trip_df['Trip (Day/Night)'] = np.where(
    (trip_df['Pickup Hour'] >= 6) &
    (trip_df['Pickup Hour'] < 18),
    'Day Trip',
    'Night Trip'
)

# Total Fare
```

```
trip_df['Total Fare'] = trip_df['fare_amount'] + trip_df['Surge Fee']

# Fare Per Mile
trip_df['Fare Per Mile'] = (
    trip_df['Total Fare'] / trip_df['trip_distance']
)

# Replace infinite values
trip_df.replace([np.inf, -np.inf], np.nan, inplace=True)

print("Feature Engineering Completed Successfully.")
```

```
Feature Engineering Completed Successfully.
```

```
trip_df.head()
```

Vehicle Type Distribution

Objective

This analysis identifies the distribution of trips across different Uber vehicle categories to understand customer preferences and demand for each service type.

```
import plotly.express as px

# Vehicle counts
vehicle_counts = (
    trip_df["Vehicle"]
    .value_counts()
    .reset_index()
)

vehicle_counts.columns = ["Vehicle", "Trips"]

fig = px.bar(
    vehicle_counts,
    x="Vehicle",
    y="Trips",
    text="Trips",
    color="Trips",
    color_continuous_scale="Greys",
    title="<b>Vehicle Type Distribution</b>"
)

fig.update_traces(
    texttemplate='%{text:,}',
    textposition='outside',
    marker_line_color='black',
    marker_line_width=1.5
)

fig.update_layout(
    template="plotly_white",
    height=550,
```

```
    width=900,
    title_x=0.5,
    font=dict(
        family="Arial",
        size=15,
        color="black"
    ),
    xaxis_title="Vehicle Type",
    yaxis_title="Number of Trips",
    coloraxis_showscale=False
)

fig.show()
```

```
# ========================================
# Payment Type Distribution
# ========================================

import plotly.express as px

# Count Payment Types
payment_counts = (
    trip_df["Payment_type"]
    .value_counts()
    .reset_index()
)

payment_counts.columns = ["Payment Type", "Trips"]

# Create Donut Chart
fig = px.pie(
    payment_counts,
    names="Payment Type",
    values="Trips",
    hole=0.68,
    color="Payment Type",
    color_discrete_map={
        "Uber Pay": "#111827",      # Navy Black
        "Cash": "#4B5563",          # Slate Gray
        "Amazon Pay": "#3B82F6",    # Blue
        "Google Pay": "#10B981"     # Emerald
    },
    title="<b>Payment Type Distribution</b>"
)

# Customize Slices
fig.update_traces(
    textposition="outside",
    textinfo="label+percent",
    pull=[0.05, 0.02, 0.02, 0.02],
    marker=dict(
        line=dict(
            color="white",
            width=3
        )
    ),
    hovertemplate=
    "<b>%{label}</b><br>" +
    "Trips: %{value:,}<br>" +
    "Percentage: %{percent}<extra></extra>"
)

# Layout
fig.update_layout(
    template="plotly_white",
    height=650,
    width=950,

    title=dict(
        text="<b>Payment Type Distribution</b>",
        x=0.5,
        font=dict(
            size=28,
            family="Arial Black",
            color="#111827"
        )
    ),

    font=dict(
        family="Arial",
        size=16,
        color="#111827"
```

```
    ),

    legend=dict(
        title="<b>Payment Type</b>",
        x=1.02,
        y=0.98,
        bgcolor="rgba(255,255,255,0)",
        borderwidth=0,
        font=dict(size=15)
    ),

    margin=dict(
        t=90,
        l=40,
        r=150,
        b=40
    )
)

fig.show()
```

```
# ========================================
# Trip Distance Distribution (0–20 Miles)
# ========================================

import plotly.express as px

fig = px.histogram(
    trip_df[trip_df["trip_distance"] <= 20],
    x="trip_distance",
    nbins=40,
    color_discrete_sequence=["#111827"],
    title="<b>Trip Distance Distribution (0–20 Miles)</b>"
)

fig.update_traces(
    marker=dict(
        line=dict(color="white", width=1)
    ),
    hovertemplate="<b>Distance</b>: %{x:.2f} miles<br><b>Trips</b>: %{y:,}<extra></extra>"
)

fig.update_layout(
    template="plotly_white",
    height=600,
    width=950,

    title=dict(
        x=0.5,
        font=dict(
            size=28,
            family="Arial Black"
        )
    ),

    xaxis_title="Trip Distance (Miles)",
    yaxis_title="Number of Trips",

    font=dict(
        family="Arial",
        size=15
    )
)

fig.show()
```

```
# ========================================
# Daily Booking Trend
# ========================================

import plotly.express as px

# Create Date Column
trip_df["Pickup Date"] = trip_df["Pickup Time"].dt.date

# Count Bookings per Day
daily_bookings = (
    trip_df.groupby("Pickup Date")
           .size()
           .reset_index(name="Bookings")
)
```

```
# Area Chart
fig = px.area(
    daily_bookings,
    x="Pickup Date",
    y="Bookings",
    title="<b>Daily Booking Trend</b>",
    line_shape="spline",
    color_discrete_sequence=["#111827"]
)

fig.update_traces(
    line=dict(width=3),
    hovertemplate=
    "<b>Date:</b> %{x}<br>" +
    "<b>Bookings:</b> %{y:,}<extra></extra>"
)

fig.update_layout(
    template="plotly_white",
    height=600,
    width=950,

    title=dict(
        x=0.5,
        font=dict(
            size=28,
            family="Arial Black"
        )
    ),

    xaxis_title="Date",
    yaxis_title="Number of Bookings",

    font=dict(
        family="Arial",
        size=15
    ),

    hovermode="x unified"
)

fig.show()
```

```
# ========================================
# Hourly Booking Trend
# ========================================

import plotly.express as px

# Extract Hour
trip_df["Pickup Hour"] = trip_df["Pickup Time"].dt.hour

# Count bookings
hourly_bookings = (
    trip_df.groupby("Pickup Hour")
           .size()
           .reset_index(name="Bookings")
)

fig = px.line(
    hourly_bookings,
    x="Pickup Hour",
    y="Bookings",
    markers=True,
    title="<b>Hourly Booking Trend</b>"
)

fig.update_traces(
    line=dict(color="#111827", width=4),
    marker=dict(
        size=9,
        color="#3B82F6",
        line=dict(color="white", width=2)
    ),
    hovertemplate="<b>Hour</b>: %{x}:00<br><b>Bookings</b>: %{y:,}<extra></extra>"
)

fig.update_layout(
    template="plotly_white",
    height=600,
    width=950,
```

```
    title=dict(
        x=0.5,
        font=dict(
            size=28,
            family="Arial Black",
            color="#111827"
        )
    ),

    xaxis=dict(
        title="Hour of Day",
        tickmode="linear",
        dtick=1
    ),

    yaxis_title="Number of Bookings",

    hovermode="x unified",

    font=dict(
        family="Arial",
        size=15
    )
)

fig.show()
```

```
# ========================================
# Fare vs Trip Distance
# ========================================

import plotly.express as px

fig = px.scatter(
    trip_df.sample(8000, random_state=42),
    x="trip_distance",
    y="fare_amount",
    color="Vehicle",
    opacity=0.65,
    title="<b>Fare Amount vs Trip Distance</b>",
    color_discrete_sequence=[
        "#111827",
        "#374151",
        "#6B7280",
        "#3B82F6",
        "#10B981"
    ]
)

fig.update_traces(
    marker=dict(size=7),
    hovertemplate=
    "<b>Distance:</b> %{x:.2f} miles<br>"
    "<b>Fare:</b> $%{y:.2f}<extra></extra>"
)

fig.update_layout(
    template="plotly_white",
    width=1000,
    height=650,

    title=dict(
        x=0.5,
        font=dict(
            size=28,
            family="Arial Black",
            color="#111827"
        )
    ),

    xaxis_title="Trip Distance (Miles)",
    yaxis_title="Fare Amount ($)",

    font=dict(size=15),

    legend_title="Vehicle Type"
)

fig.show()
```

```
# ========================================
# Correlation Heatmap
# ========================================

import plotly.express as px

# Select numerical columns
corr = trip_df[
    [
        "passenger_count",
        "trip_distance",
        "fare_amount",
        "Surge Fee"
    ]
].corr()

fig = px.imshow(
    corr,
    text_auto=".2f",
    color_continuous_scale=[
        "#F3F4F6",
        "#D1D5DB",
        "#9CA3AF",
        "#4B5563",
        "#111827"
    ],
    title="<b>Correlation Heatmap</b>"
)

fig.update_layout(
    template="plotly_white",
    width=800,
    height=700,

    title=dict(
        x=0.5,
        font=dict(
            size=28,
            family="Arial Black",
            color="#111827"
        )
    ),

    font=dict(
        family="Arial",
        size=15
    ),

    coloraxis_colorbar=dict(
        title="Correlation"
    )
)

fig.show()
```

```
# ========================================
# Top 10 Pickup Locations
# ========================================

import plotly.express as px

pickup_locations = (
    trip_df["Pickup Location"]
    .value_counts()
    .head(10)
    .sort_values()
    .reset_index()
)

pickup_locations.columns = ["Pickup Location", "Trips"]

fig = px.bar(
    pickup_locations,
    x="Trips",
    y="Pickup Location",
    orientation="h",
    text="Trips",
    color="Trips",
    color_continuous_scale="Blues",
    title="<b>Top 10 Pickup Locations</b>"
)

fig.update_traces(
    texttemplate="%{text:,}",
    textposition="outside",
    marker_line_color="white",
    marker_line_width=1.5,
    hovertemplate="<b>%{y}</b><br>Trips: %{x:,}<extra></extra>"
)

fig.update_layout(
    template="plotly_white",
    height=650,
    width=950,

    title=dict(
        text="<b>Top 10 Pickup Locations</b>",
        x=0.5,
        font=dict(
            size=28,
            family="Arial Black",
            color="#111827"
        )
    ),

    xaxis_title="Total Trips",
    yaxis_title="Pickup Location",

    font=dict(
        family="Arial",
        size=15
    ),

    coloraxis_showscale=False
)

fig.show()
```

```
# ========================================
# Booking Heatmap (Day vs Hour)
# ========================================

import plotly.express as px

# Create Pivot Table
heatmap_data = trip_df.pivot_table(
```

```
    index="Day Name",
    columns="Pickup Hour",
    values="Trip ID",
    aggfunc="count"
)

# Reorder Days
day_order = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday"
]

heatmap_data = heatmap_data.reindex(day_order)

# Plot Heatmap
fig = px.imshow(
    heatmap_data,
    text_auto=True,
    aspect="auto",
    color_continuous_scale="Blues",
    title="<b>Booking Demand by Hour and Day</b>"
)

fig.update_layout(
    template="plotly_white",
    width=1000,
    height=600,

    title=dict(
        x=0.5,
        font=dict(
            size=28,
            family="Arial Black",
            color="#111827"
        )
    ),

    xaxis_title="Hour of Day",
    yaxis_title="Day of Week",

    font=dict(
        family="Arial",
        size=15
    )
)

fig.show()
```

Insight

- Booking demand varies significantly throughout the week.
- Peak booking periods can be easily identified by darker cells.
- This visualization helps optimize driver allocation during high-demand hours.
- Weekends and evening hours generally experience higher ride requests.

```
# ========================================
# Average Fare by Vehicle Type
# ========================================

fig = px.box(
    trip_df,
    x="Vehicle",
    y="fare_amount",
```

```
        color="Vehicle",
        color_discrete_sequence=[
            "#111827",
            "#374151",
            "#6B7280",
            "#3B82F6",
            "#10B981"
        ],
        title="<b>Fare Distribution by Vehicle Type</b>"
    )

    fig.update_layout(
        template="plotly_white",
        width=950,
        height=600,

        title=dict(
            x=0.5,
            font=dict(
                size=28,
                family="Arial Black"
            )
        ),

        xaxis_title="Vehicle Type",
        yaxis_title="Fare Amount ($)",

        font=dict(size=15),

        showlegend=False
    )

    fig.show()
```

Insight

- Premium vehicle categories generally have higher median fares.
- Some vehicle types exhibit a wider fare distribution, indicating greater pricing variability.
- Outliers highlight unusually expensive trips, possibly influenced by surge pricing or long-distance rides.

Executive Summary

This project analyzes **103,728 Uber trips** to uncover trends in customer behavior, booking demand, payment preferences, trip characteristics, and location-based activity.

Using **Python**, **Pandas**, **Plotly**, and **Power BI**, the analysis transforms raw operational data into actionable business insights through exploratory data analysis and interactive dashboards.

The study focuses on identifying peak demand periods, understanding customer preferences, evaluating fare patterns, and providing recommendations that support operational efficiency and strategic decision-making.

Business Insights

Customer Behavior

- Uber Pay is the most frequently used payment method, indicating a strong preference for digital payments.
- Most rides involve a single passenger, suggesting individual commuting is the dominant travel pattern.

Trip Characteristics

- The majority of Uber trips cover relatively short distances.
- Fare amount increases with trip distance, showing a clear positive relationship.
- Surge pricing contributes to higher fares during periods of increased demand.

Booking Trends

- Ride demand fluctuates throughout the day, with noticeable peak booking hours.
- Daily booking patterns reveal variations across weekdays and weekends.
- Booking activity changes over time, highlighting demand spikes on specific dates.

Location Analysis

- A small number of pickup locations generate a significant portion of total bookings.
- Certain drop-off locations consistently experience higher passenger traffic.
- Location-based demand can be leveraged for better driver positioning.

Operational Insights

- Peak demand periods require increased driver availability.
- High-demand locations should receive priority for driver allocation.
- Digital payment adoption provides opportunities for loyalty programs and promotional campaigns.

Business Recommendations

Improve Fleet Allocation

Deploy additional drivers during peak booking hours and in high-demand pickup locations to reduce passenger waiting time.

Optimize Surge Pricing

Use historical demand patterns to implement dynamic surge pricing strategies while maintaining customer satisfaction.

Promote Digital Payments

Encourage customers to use digital payment methods through cashback offers and loyalty rewards.

Location-Based Driver Positioning

Use pickup hotspot analysis to strategically position drivers before demand peaks.

Customer Experience

Improve ride availability during weekends and high-demand hours to increase customer retention.

Data-Driven Decision Making

Continuously monitor booking trends, payment behavior, and trip characteristics to support operational planning.

Conclusion

This project demonstrates the complete workflow of a business intelligence solution, starting from raw data exploration and ending with actionable insights.

Through exploratory data analysis and interactive dashboard development, key trends in booking demand, customer behavior, payment preferences, vehicle usage, and location patterns were identified.

The combination of Python-based analytics and Power BI visualization provides an effective framework for understanding operational performance and supporting data-driven business decisions.

The techniques used in this project can be extended to demand forecasting, customer segmentation, revenue prediction, and fleet optimization in future work.

```
trip_df.head()
```

```
fig.write_image("vehicle_distribution.png", scale=3)
```

```
!pip install -q kaleido
```

```
fig.write_image("vehicle_distribution.png", scale=3)
```

```
!pip install -q kaleido
```